

ECE522

COMPUTER ORGANIZATION

SEMESTER OCT 2025 – MAC 2026

ASSIGNMENT 2

PREPARED BY:

NO	NAME	STUDENT ID
1	ANIS SUHANA BINTI MOHD AZMI	2023816852
2	DAHLIA FATEHA BINTI AZIZI	2023262198

PREPARED FOR:

PROFESOR MADYA TS. DR. NORLIZA BT MOHAMAD ZAINI

TABLE CONTENTS

1. Introduction....3
1.1 Background of Arithmetic Logic Unit (ALU).... 3
1.2 Objectives of the Assignment.... 4
1.3 Scope of Work.... 4
2. ALU Specification and Design Overview....5
2.1 ALU Inputs and Outputs.... 5
2.2 Operation Mapping and Selection Logic....5
2.3 1-bit ALU Slice Architecture....6
3. Question 1: Logic Analysis and Operation Selection.... 6
3.1 Operation Specification and Behaviour (Q1a).... 6
3.1.1 XOR Operation.... 6
3.1.2 ADD Operation....7
3.1.3 AND Operation....8
3.1.4 XNOR Operation....9
3.1.5 Role of cin and cout.... 10
3.1.6 Operation Selection using (i, j)....10
3.1.6.1 Multiplexer Operation.... 11
3.2 Complete Truth Table of 1-bit ALU (Q1b).... 12
3.3 Boolean Expression Derivation and Simplification (Q1c).... 13
3.3.1 Boolean Expression for Output y.... 13
3.3.2 Boolean Expression for Carry-out cout....14
3.3.3 Simplification using Boolean Algebra / Karnaugh Map....14
4. Question 2: Verilog HDL Module Design and ALU Integration....15
4.1 Reusable ALU Operation Modules (Q2a).... 15
4.1.1 XOR Module.... 15
4.1.2 AND Module....15
4.1.3 XNOR Module....16
4.1.4 Full Adder Module....16
4.2 Operation Selection Modules (Q2b).... 17
4.2.1 4-to-1 Multiplexer Design....17
4.2.2 2-to-4 Decoder Design....18
4.3 Top-Level 1-bit ALU Integration (Q2c).... 20
4.3.1 Multiplexer-Based ALU Architecture....20
4.3.2 Decoder-Based ALU Architecture....21
5. Question 3: Simulation, Verification, and Evaluation....22
5.1 Waveform-Based Verification (Q3a).... 22
5.1.1 Multiplexer-Based ALU Waveforms.... 22
5.1.2 Decoder-Based ALU Waveforms....23

5.2 Text-Based Verification and Self-Checking Testbench (Q3b)..... 24
5.2.1 Verification Methodology.....24
5.2.2 Comparison with Truth Table..... 24
5.2.3 Pass/Fail Analysis.....28
6. Results and Discussion..... 30
6.1 Functional Correctness Analysis..... 30
6.2 Comparison of Multiplexer vs Decoder-Based Architecture..... 30
6.3 Performance and Hardware Efficiency Considerations..... 30
7. Conclusion..... 31
8. Appendix..... 32
A. Testbench Codes**.....32**

1. Introduction

1.1 Background of Arithmetic Logic Unit (ALU)

The Arithmetic Logic Unit (ALU) is the core computational part of a CPU, performing arithmetic and logical operations, with its concept pioneered by John von Neumann, who is a mathematician and scientist, in 1945 for the EDVAC computer. In the early mechanical computers, such as the abacus and Pascal's machin,e can be considered as primitive versions of the ALU. Early Arithmetic Logic Units processed data one bit at a time due to high costs and power consumption. In 1948, the Whirlwind computer was built, and it was one of the first computers that featured 16 single-bit ALU circuits for 16-bit words. By the late 1960s and early 1970s, ALUs evolved into integrated circuits, such as the 8-bit Fairchild 3800 and 4-bit Texas Instruments 74181. As transistors miniaturized, ALUs were integrated into microprocessors like IBM's PALM, which utilized an 8-bit ALU to perform 16-bit operations in two passes. This ongoing shrinkage of transistors has allowed ALUs to become more complex and faster.

An ALU is a combinational circuit that produces output based solely on given input data, interacting with the processor's control unit, memory unit, and input/output devices. It receives an instruction code from the control unit that includes an operation code (opcode) and one or more operands. The ALU outputs the result of operations, which is stored in memory, along with status signals indicating the operation's success, negativities, overflow, or carries. The design of the ALU is crucial for overall CPU performance, influencing speed and efficiency. Variations exist in ALU design across processors, with some employing multiple ALUs or special units for floating-point calculations, and they are also essential components in GPUs for rapid calculations in graphics generation.

1.2 Objectives of the Assignment

1. To analyze the behaviour of a 1-bit Arithmetic Logic Unit (ALU) by studying basic logic and arithmetic operations.

2. To design a decoder-controlled 1-bit ALU slice using Boolean logic and Verilog HDL for embedded applications.

3. To verify the ALU functionality through truth table analysis and simulation results.

1.3 Scope of Work

The scope of work for this assignment involves analyzing the operation of a 1-bit ALU slice used in an embedded microcontroller system. This includes defining the behaviour of XOR, ADD, AND, and XNOR operations, and examining the role of carry-in and carry-out signals in arithmetic computation. A complete truth table is constructed based on all possible combinations of input and control signals, followed by the derivation of simplified Boolean expressions for the ALU outputs. The design is then implemented using Verilog HDL with a decoder-based control mechanism to ensure correct operation selection. Finally, the ALU design is verified through simulation and waveform analysis to confirm that the outputs match the expected logic behaviour.

2. ALU Specification and Design Overview

2.1 ALU Inputs and Outputs

The status inputs allow the ALU to receive additional information while conducting an operation. Typically, this is a single "carry-in" bit that represents the carry-out from a prior ALU operation. A general-purpose Arithmetic Logic Unit's (ALU) status outputs are made up of different separate signals that offer

more details on the outcomes of the present operation. Important status indicators consist of:

1. Carry-out: Indicates the overrun from a binary shift, the borrow from a subtraction, or the carry from an addition.
2. Zero: This means that the operand Y's bits are all zero.
3. Negative: Indicates that arithmetic processes have produced a negative result.
4. Overflow: This indicates that the outcome is greater than the operand's numerical range.
5. Parity: Denotes whether Y has an even or odd number of bits set to one.

2.2 Operation Mapping and Selection Logic

The ALU supports four operations, which are AND, OR, ADD, and XOR/XNOR. The two-bit control signal (j, i) determines which operations are performed. Only one operation is carried out at a time because each combination of the control inputs initiates a distinct action. The following illustrates how the control signals are mapped to the appropriate ALU operations:

j	i	Selected operation
0	0	AND
0	1	OR
1	0	ADD
1	1	XOR/XNOR

A 4-to-1 multiplexer or a 2-to-4 decoder can be used to implement the selection logic. Each decoder output in the decoder-based method permits a particular operating circuit. All operation outputs are produced in parallel in the multiplexer-based method, and the multiplexer uses the control signals to determine which output is needed.

2.3 1-bit ALU Slice Architecture

The 1-bit ALU slice architecture consists of parallel logic blocks that independently compute each supported operation. These include logic gates for AND and OR operations, an XOR/XNOR block for exclusive logic operations, and a full adder block for arithmetic addition. All operation outputs are fed into a selection unit implemented using either a decoder with enabling logic or a multiplexer. The selected output is then forwarded as the final ALU output *y*. The carry-out signal is generated exclusively by the full adder block and propagated to the next ALU slice during multi-bit operation. This modular 1-bit slice design allows multiple ALU slices to be cascaded easily to form a wider ALU, making the architecture scalable and suitable for embedded processor applications.

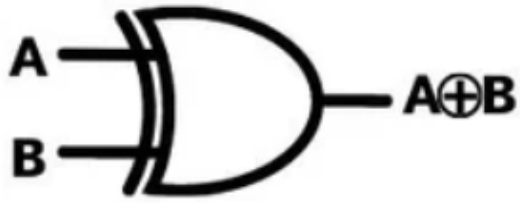
3. Question 1: Logic Analysis and Operation Selection

3.1 Operation Specification and Behaviour (Q1a)

3.1.1 XOR Operation

Boolean expressions : $y_{xor} = a \oplus b$

Logic circuit :



Truth table :

A	B	A and B
0	0	0

0	1	1
1	0	1
1	1	0

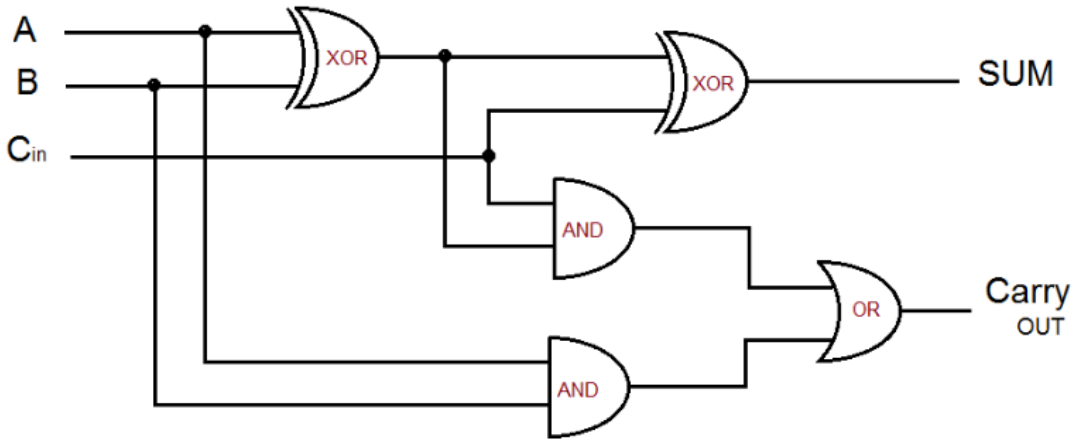
3.1.2 ADD Operation

Boolean expression :

Sum : $y_{add} = a \oplus b$

Carry-out : $cout = ab + acin + bcin$

Logic circuit :



Truth table :

A	B	Cin	SUM	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1

1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

3.1.3 AND Operation

Boolean expression : $y_{and} = a \cdot b$

Logic circuit :



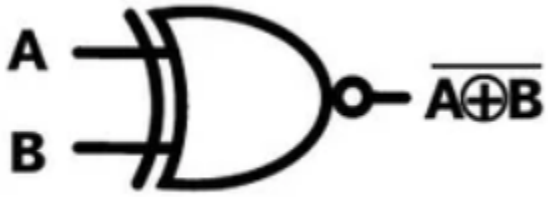
Truth table :

A	B	A and B
0	0	0
0	1	0
1	0	0
1	1	1

3.1.4 XNOR Operation

Boolean expression : $y_{xnor} = \overline{a \oplus b}$

Logic circuit :



Truth table :

A	B	A and B
0	0	1
0	1	0
1	0	0
1	1	1

3.1.5 Role of cin and cout

The 1-bit ALU supports arithmetic operations via the signals cin (carry-in) and cout (carry-out). The carry-in signal cin is only used during the ADD operation, the ALU may execute full adder functionality. This allows for proper carry propagation when multiple 1-bit ALU slices are cascaded to create a multi-bit ALU. For logic operations such as AND, OR, XOR, and XNOR, the cin signal has no effect on the output and is thus handled as a don't care input. The carry-out signal cout specifies whether a carry occurs during the ADD operation. It stays at logic 0 for all other operations and is only asserted when addition is carried out. This ensures correct arithmetic behavior and prevents unintentional carry propagation during logic operations.

3.1.6 Operation Selection using (i, j)

2 - 4 truth Decoder Truth table

j	i	D0	D1	D2	D3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

D0 activates the XOR operation while D1 activates the ADD operation. Next is D3, which will activatethe AND operation, and lastly, D3 will activate the XNOR operation.

3.1.6.1 Multiplexer Operation

This ALU design uses a 4-to-1 multiplexer in place of the decoder-based operation selection. The control signals (j, i) serve as the select lines, and the outputs of the four ALU operations are directly connected to the multiplexer inputs. The final ALU output, y, is represented by the multiplexer's output. The input of the multiplexer is defined as follows:

- I0 is defined as the AND operation output
- I1 is defined as the OR operation output
- 12 is defined as the ADD operation output
- I3 is defined as the XOR or XNOR operation output

The table below will show the multiplexer selection table. It shows that the select lines (*j,i*) is uniquely determine which ALU operation output is forwarded to the ALU output

j	i	Selected MUX input	Active ALU operation
0	0	I0	XOR
0	1	I1	ADD
1	0	I2	AND
1	1	I3	XNOR

Decoder-based enabling activates only one ALU operation by enabling its logic circuit, whereas multiplexer-based selection picks a single operation output from numerous inputs. Both systems ensure that only one operation result is sent to the ALU output at any given moment, hence they are functionally equal for ALU operation selection.

3.2 Complete Truth Table of 1-bit ALU (Q1b)

a	b	cin	j	i	y	cout	Operation
0	0	0/X	0	0	0	0	XOR
0	1	0/X	0	0	1	0	
1	0	0/X	0	0	1	0	
1	1	0/X	0	0	0	0	
0	0	0	0	1	0	0	ADD
0	1	0	0	1	1	0	
1	0	0	0	1	1	0	
1	1	0	0	1	0	1	

0	0	1	0	1	1	0	
0	1	1	0	1	0	1	
1	0	1	0	1	0	1	
1	1	1	0	1	1	1	
0	0	0/X	1	0	0	0	AND
0	1	0/X	1	0	0	0	
1	0	0/X	1	0	0	0	
1	1	0/X	1	0	1	0	
0	0	0/X	1	1	1	0	XNOR
0	1	0/X	1	1	0	0	
1	0	0/X	1	1	0	0	
1	1	0/X	1	1	1	0	

3.3 Boolean Expression Derivation and Simplification (Q1c)

3.3.1 Boolean Expression for Output y

The ALU output (y) is decided by the control signals (j, i), based on the complete 1-bit ALU truth table generated. Each operation results in a different output expression.

AND operation :

$$y_{AND} = a \cdot b$$

OR operation :

$$y_{OR} = a + b$$

ADD operation :

$$y_{ADD} = a \oplus b \oplus cin$$

XOR/XNOR operation :

$$y_{XOR} = a \oplus b$$

$$y_{XNOR} = \overline{a \oplus b}$$

Using decoder or multiplexer-based selection, the whole ALU output (y) can be stated in Sum of Products (SOP) form:

$$y = D_0(a \cdot b) + D_1(a + b) + D_2(a \oplus b \oplus cin) + D_3(a \oplus b)$$

Decoder D0 to D3 are the decoder outputs corresponding to the control inputs (*j,i)*

3.3.2 Boolean Expression for Carry-out cout

The truth table in 3.2 shows that the carry-out signal is valid only during the ADD operation. For all other operations, the carry-out is set to logic zero. The Boolean expression for carry-out during addition (full adder) is as follows:

$$cout = ab + acin + bcin$$

Including the operation selection condition, the carry-out expression becomes:

$$cout = D_2(ab + acin + bcin)$$

This ensures that coutcoutcout is active only when the ADD operation is selected and remains 0 for all non-ADD operations, as required by the assignment specification

3.3.3 Simplification using Boolean Algebra / Karnaugh Map

The ALU output (y) is already optimized by isolating operations and selecting them using decoder outputs. Since only one decoder output is active at any given moment, no further Boolean reduction is necessary for the selection logic. This method eliminates duplicate logic while ensuring proper operation isolation. For the ADD operation, the sum result is already in its most basic form:

$$y_{ADD} = a \oplus b \oplus cin$$

The carry-out expression is a simplified SOP form derived directly from the entire adder truth table. The Karnaugh map analysis reveals that no additional simplification is achievable without changing the logical behavior. The sum of the result as follow :

$$cout = ab + acin + bcin$$

The final simplified expression would be as follow:

$$y = D_0(a \cdot b) + D_1(a + b) + D_2(a \oplus b \oplus cin) + D_3(a \oplus b)$$

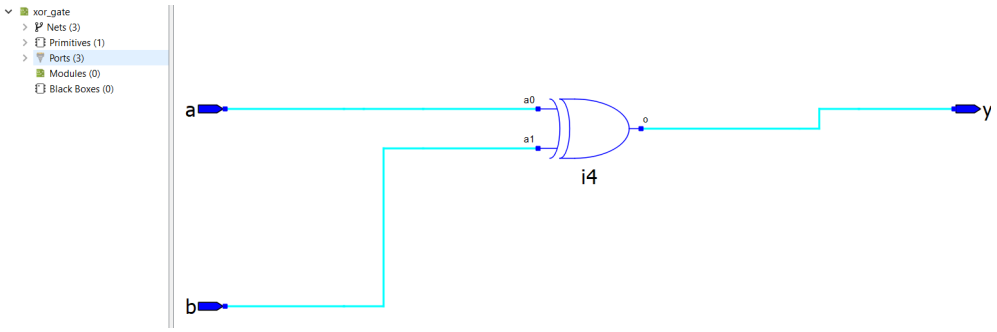
$$cout = D_2(a \oplus b \oplus cin)$$

4. Question 2: Verilog HDL Module Design and ALU Integration

4.1 Reusable ALU Operation Modules (Q2a)

4.1.1 XOR Module

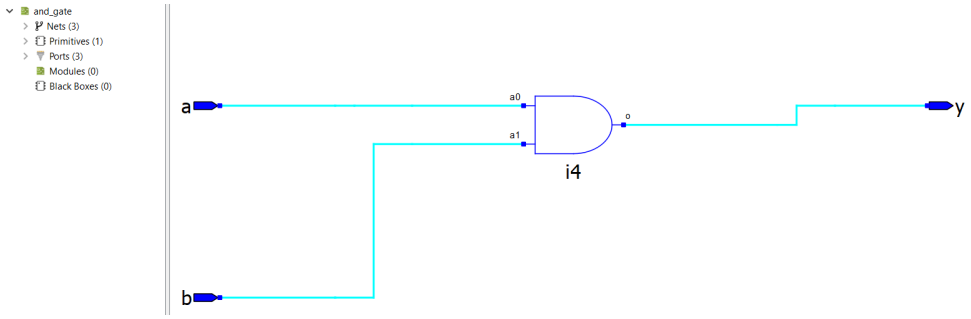
```
1  module xor_gate (input a, b, output y);
2      assign y = a ^ b;
3  endmodule
```



This module produces a high output only when the two inputs have different logic values. It is a key component in arithmetic operations such as adders to produce the sum bit.

4.1.2 AND Module

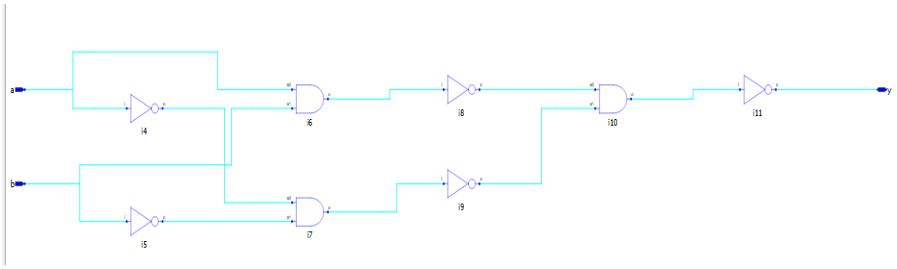
```
1  module and_gate (input a, b, output y);
2      assign y = a & b;
3  endmodule
```



This module performs a logical multiplication operation, where the output is high only if all its inputs are high. It is often used as a signal filter or in the construction of an adder circuit to produce a carry bit.

4.1.3 XNOR Module

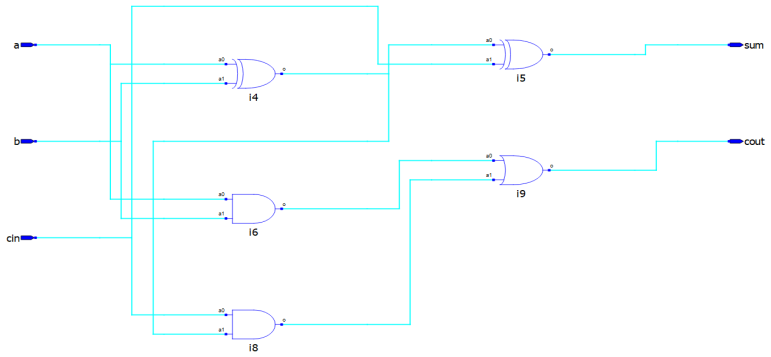
```
1  module xnor_gate (input a, b, output y);
2  wire w1, w2, w3, w4, w5, w6;
3  assign w1 = a & b;
4  assign w2 = ~a;
5  assign w3 = ~b;
6  assign w4 = w2 & w3;
7  assign w5 = ~w1;
8  assign w6 = ~w4;
9  assign y = ~(w5 & w6);
10 endmodule
```



This circuit functions as an equality detector by producing a high output when both inputs are equal. It is the inverse of the XOR gate and is important in digital comparison circuits.

4.1.4 Full Adder Module

```
1  module full_adder (input a, b, cin, output sum, cout);
2      assign sum = a ^ b ^ cin;
3      assign cout = (a & b) | (cin & (a ^ b));
4  endmodule
```



This module is capable of summing three input bits, which include two data bits and one carry-in bit. It produces two outputs, which are the sum and the carry-out to be connected to the next stage.

4.2 Operation Selection Modules (Q2b)

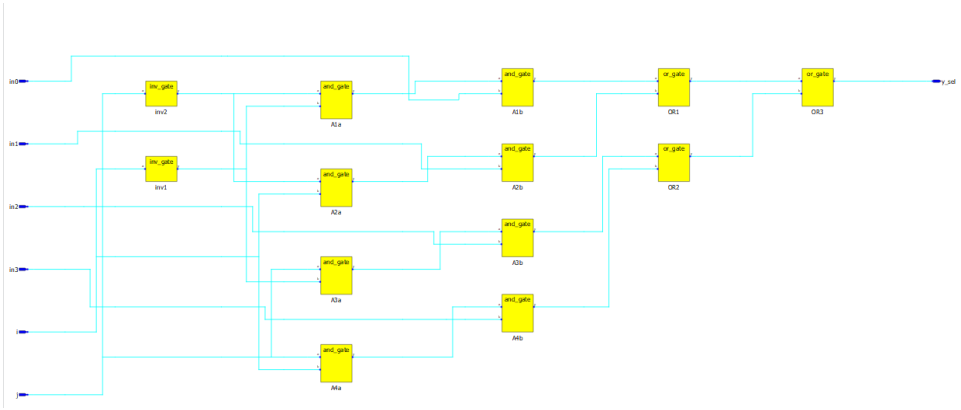
4.2.1 4-to-1 Multiplexer Design

```
module multiplexer (input  in0, in1, in2, in3, i, j, output y_sel);
    wire w1, w2;
    wire D0_int, D1_int, D2_int, D3_int;
    wire D0, D1, D2, D3;

    inv_gate inv1 (i, w1);
    inv_gate inv2 (j, w2);

    and_gate A1a (w2, w1, D0_int);
    and_gate A1b (D0_int, in0, D0);
    and_gate A2a (w2, i, D1_int);
    and_gate A2b (D1_int, in1, D1);
    and_gate A3a (j, w1, D2_int);
    and_gate A3b (D2_int, in2, D2);
    and_gate A4a (j, i, D3_int);
    and_gate A4b (D3_int, in3, D3);

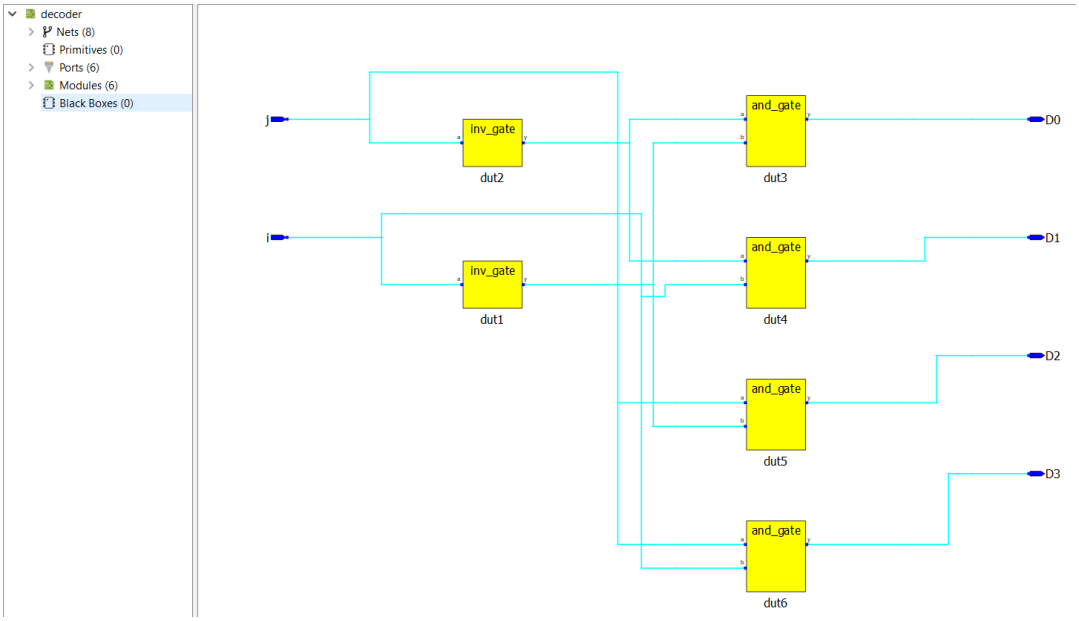
    wire or1_out, or2_out;
    or_gate OR1 (D0, D1, or1_out);
    or_gate OR2 (D2, D3, or2_out);
    or_gate OR3 (or1_out, or2_out, y_sel);
endmodule
```



This design allows the selection of one of four data input lines to be sent to a single output line. The selection of the input is controlled entirely by the combination of the values on two select signal bits (select lines).

4.2.2 2-to-4 Decoder Design

```
1  module decoder (input j, i, output D0, D1, D2, D3);
2      wire w1, w2;
3      inv_gate dut1(i, w1);
4      inv_gate dut2(j, w2);
5      and_gate dut3 (w2, w1, D0); // j=0, i=0 - XOR enable
6      and_gate dut4 (w2, i,  D1); // j=0, i=1 - ADD enable
7      and_gate dut5 (j,  w1, D2); // j=1, i=0 - AND enable
8      and_gate dut6 (j,  i,  D3); // j=1, i=1 - XNOR enable
9  endmodule
```

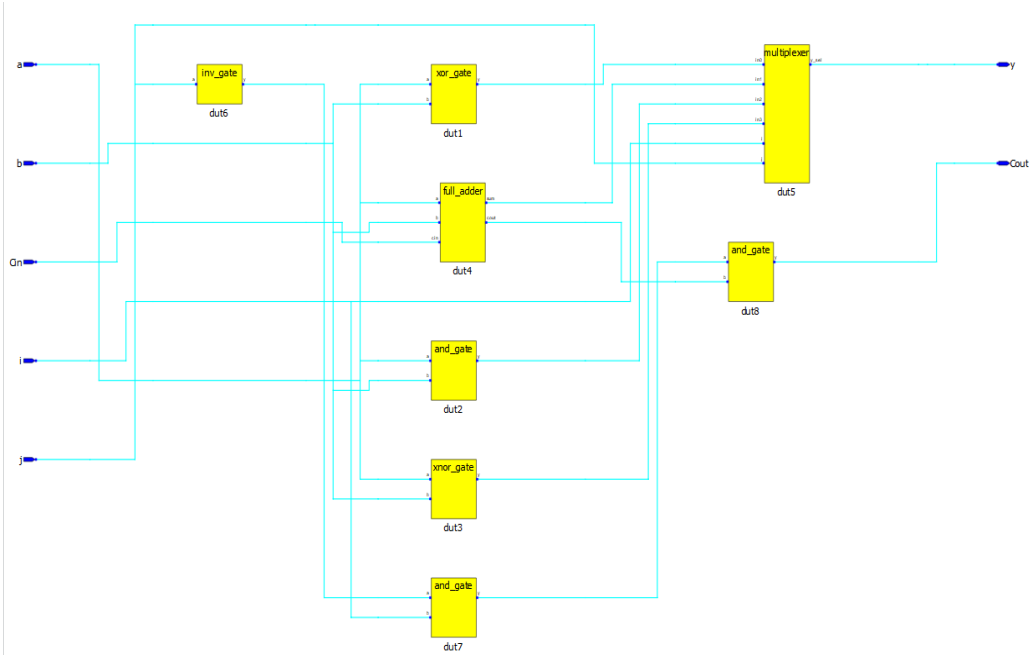


This circuit converts a two-bit binary input code into one of four uniquely active outputs. It functions to translate the code information into a specific selection signal in a digital system.

4.3 Top-Level 1-bit ALU Integration (Q2c)

4.3.1 Multiplexer-Based ALU Architecture

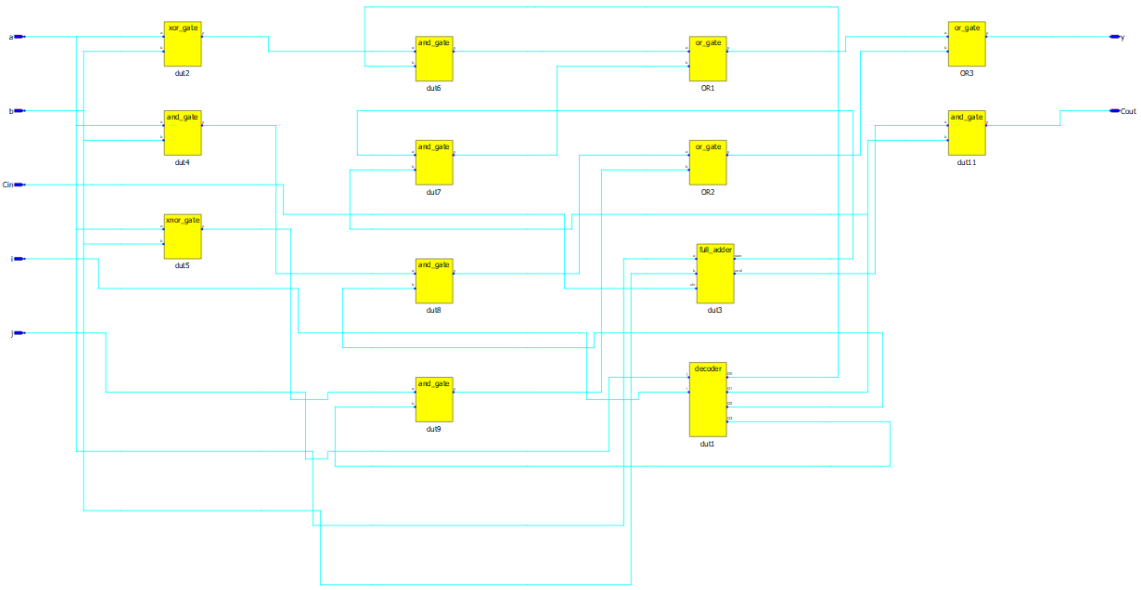
```
module MultiplexerALU (input  a, b, Cin, i, j, output y, output Cout);
    wire w1, w2, w3, w4, w5, w6, w7;
    xor_gate   dut1 (a, b, w1);
    and_gate   dut2 (a, b, w2);
    xnor_gate  dut3 (a, b, w3);
    full_adder dut4 (a, b, Cin, w4, w5);
    multiplexer dut5 (
        .in0(w1),
        .in1(w4),
        .in2(w2),
        .in3(w3),
        .i(i),
        .j(j),
        .y_sel(y)
    );
    inv_gate dut6 (j, w6);
    and_gate dut7 (w6, i, w7);
    and_gate dut8 (w7, w5, Cout);
endmodule
```



This architecture uses a multiplexer to select the desired result of a logical or arithmetic operation based on control signals. It is an efficient approach to managing the flow of data between various functional units in the processor.

4.3.2 Decoder-Based ALU Architecture

```
module DecoderALU (input  a, b, Cin, i, j, output y, Cout);
    wire w1, w2, w3, w4, w5, w6, w7, w8, w9, w10, w11, w12, w13, w14, w15;
    decoder dut1 (
        .i(i),
        .j(j),
        .D0(w10),
        .D1(w11),
        .D2(w12),
        .D3(w13)
    );
    xor_gate   dut2 (a, b, w1);
    full_adder dut3 (a, b, Cin, w2, w3);
    and_gate   dut4 (a, b, w4);
    xnor_gate  dut5 (a, b, w5);
    and_gate dut6 (w1, w10, w6);
    and_gate dut7 (w2, w11, w7);
    and_gate dut8 (w4, w12, w8);
    and_gate dut9 (w5, w13, w9);
    or_gate OR1 (w6, w7, w14);
    or_gate OR2 (w8, w9, w15);
    or_gate OR3 (w14, w15, y);
    and_gate dut11 (w3, w11, Cout);
endmodule
```



This architecture uses a decoder to specifically activate certain operational units based on the given instruction code. This method ensures that only the necessary circuitry is operational for each operation cycle to reduce data errors.

5. Question 3: Simulation, Verification, and Evaluation

5.1 Waveform-Based Verification (Q3a)

5.1.1 Multiplexer-Based ALU Waveforms

testbench.sv

```
`timescale 1ns/100ps
module testbench;
    reg a, b, Cin, i, j;
    wire y, Cout;
    MultiplexerALU dut (
        .a(a), .b(b), .Cin(Cin),
        .i(i), .j(j),
        .y(y), .Cout(Cout)
    );
    initial begin
        $dumpfile("mux_alu.vcd");
        $dumpvars(0, testbench);

        a=0; b=0; Cin=0; i=0; j=0; #10; // XOR
        a=0; b=1; #10;
        a=1; b=0; #10;
        a=1; b=1; #10;

        a=0; b=0; Cin=0; i=0; j=1; #10; // ADD
        a=0; b=1; Cin=1; #10;
        a=1; b=0; Cin=1; #10;
        a=1; b=1; Cin=0; #10;

        a=0; b=0; i=1; j=0; #10;  // AND
        a=0; b=1; #10;
        a=1; b=0; #10;
        a=1; b=1; #10;

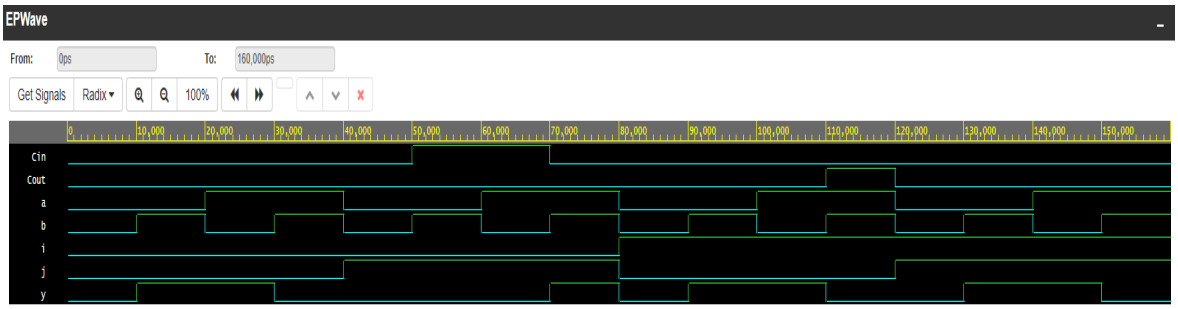
        a=0; b=0; i=1; j=1; #10; // XNOR
        a=0; b=1; #10;
        a=1; b=0; #10;
        a=1; b=1; #10;

        $finish;
    end
endmodule
```

design.sv

```
`timescale 1ns/100ps
`include "and_gate.v"
`include "or_gate.v"
`include "inv_gate.v"
`include "xor_gate.v"
`include "xnor_gate.v"
`include "full_adder.v"
`include "multiplexer.v"

module MultiplexerALU (input  a, b, Cin, i, j, output y, output Cout);
    wire w1, w2, w3, w4, w5, w6, w7;
    xor_gate   dut1 (a, b, w1);
    and_gate   dut2 (a, b, w2);
    xnor_gate  dut3 (a, b, w3);
    full_adder dut4 (a, b, Cin, w4, w5);
    multiplexer dut5 (
        .in0(w1),
        .in1(w4),
        .in2(w2),
        .in3(w3),
        .i(i),
        .j(j),
        .y_sel(y)
    );
    inv_gate dut6 (j, w6);
    and_gate dut7 (w6, i, w7);
    and_gate dut8 (w7, w5, Cout);
endmodule
```



This waveform display shows the output results that change according to the selection of the select signal on the multiplexer for each arithmetic and logical operation. The effectiveness of the circuit can be verified when the output matches the expected value based on the input data A and B for each simulation time interval.

5.1.2 Decoder-Based ALU Waveforms

testbench.sv

```
`timescale 1ns/100ps

module testbench;
    reg a, b, Cin;
    reg i, j;
    wire y, Cout;
    DecoderALU DUT (
        .a(a),
        .b(b),
        .Cin(Cin),
        .i(i),
        .j(j),
        .y(y),
        .Cout(Cout)
    );

    initial begin
        $dumpfile("decoderALU_waveform.vcd");
        $dumpvars(0, testbench);
        $display("time\ta b Cin i j | y Cout");
        $monitor("%0dns\t%b %b %b %b %b | %b %b", $time, a, b, Cin, i, j,
y, Cout);
        a=0; b=0; Cin=0; i=0; j=0; #10;
        a=0; b=1; Cin=0; i=0; j=0; #10;
        a=1; b=0; Cin=0; i=0; j=0; #10;
        a=1; b=1; Cin=0; i=0; j=0; #10;

        a=0; b=0; Cin=0; i=0; j=1; #10;  // ADD
        a=0; b=1; Cin=0; i=0; j=1; #10;
        a=1; b=0; Cin=1; i=0; j=1; #10;
        a=1; b=1; Cin=1; i=0; j=1; #10;

        a=0; b=0; Cin=0; i=1; j=0; #10;  // AND
        a=0; b=1; Cin=0; i=1; j=0; #10;
        a=1; b=0; Cin=0; i=1; j=0; #10;
        a=1; b=1; Cin=0; i=1; j=0; #10;

        a=0; b=0; Cin=0; i=1; j=1; #10;  // XNOR
        a=0; b=1; Cin=0; i=1; j=1; #10;
        a=1; b=0; Cin=0; i=1; j=1; #10;
        a=1; b=1; Cin=0; i=1; j=1; #10;

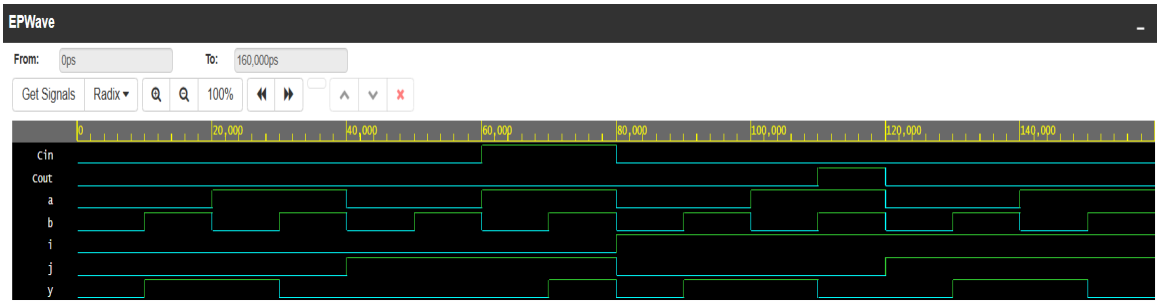
        $finish;
    end
endmodule
```

design.sv and_gate.v decoder.v DecoderALU.v full_adder.v inv_gate.v multiplexer.v xor_gate.v

```
`timescale 1ns/100ps
`include "and_gate.v"
`include "or_gate.v"
`include "inv_gate.v"
`include "xor_gate.v"
`include "xnor_gate.v"
`include "full_adder.v"
`include "decoder.v"

module DecoderALU (input  a, b, Cin, i, j, output y, Cout);
    wire w1, w2, w3, w4, w5, w6, w7, w8, w9, w10, w11, w12, w13, w14, w15;
    decoder dut1 (
        .i(i),
        .j(j),
        .D0(w10),
        .D1(w11),
        .D2(w12),
        .D3(w13)
    );
    xor_gate   dut2 (a, b, w1);
    full_adder dut3 (a, b, Cin, w2, w3);
    and_gate   dut4 (a, b, w4);
    xnor_gate  dut5 (a, b, w5);
    and_gate dut6 (w1, w10, w6);
    and_gate dut7 (w2, w11, w7);
    and_gate dut8 (w4, w12, w8);
    and_gate dut9 (w5, w13, w9);
    or_gate OR1 (w6, w7, w14);
    or_gate OR2 (w8, w9, w15);
    or_gate OR3 (w14, w15, y);
    and_gate dut11 (w3, w11, Cout);
endmodule

```



This simulation shows how the input signal is decoded to activate specific ALU functions to produce the correct output. The timeline shows the stable output status transitions each time a new instruction code is input into the decoder unit.

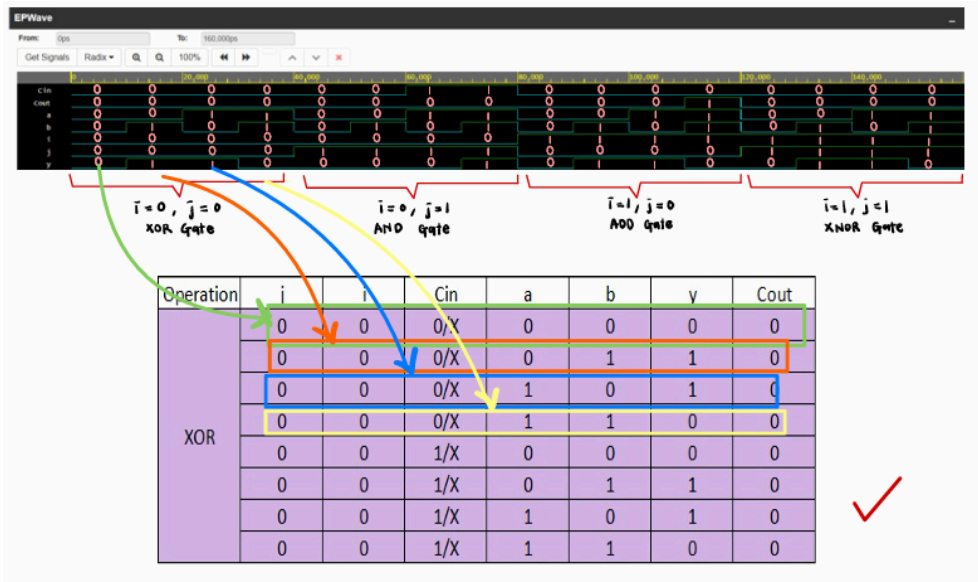
5.2 Text-Based Verification and Self-Checking Testbench (Q3b)

5.2.1 Verification Methodology

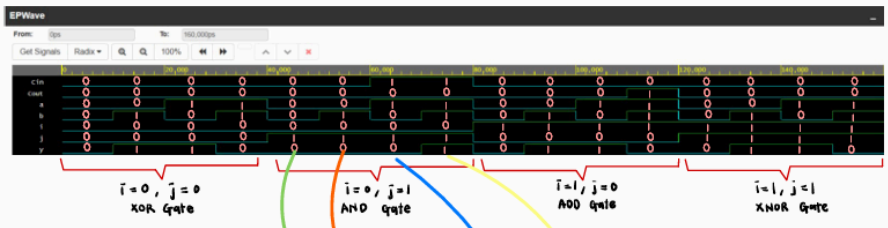
The verification methodology for Q3(b) involves ensuring that both the Multiplexer-based and Decoder-based 1-bit ALUs produce correct outputs for all possible input combinations. The testbench declares inputs a, b, Cin, i, and j, and monitors the outputs y (the ALU result) and Cout (carried out for the ADD operation). During simulation, the $display and $monitor statements are used to print the results in a table format for easy observation. In addition, a self-checking mechanism is implemented in the testbench to automatically compare the simulation output with the expected results from the truth table. If the output matches the expected value, it is marked as a pass, otherwise, a fail is recorded, clearly indicating any functional discrepancies.

5.2.2 Comparison with Truth Table

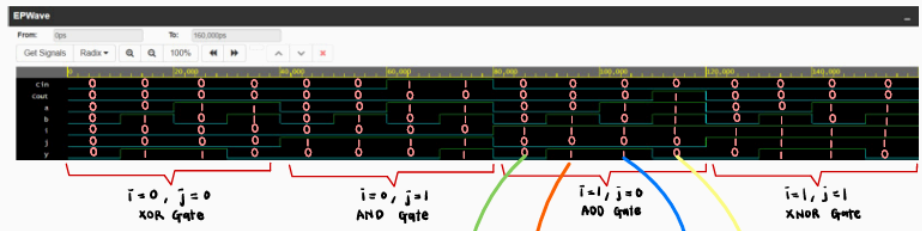
Comparisan Multiplexer-Based ALU Waveforms with Truth Table



Operation	j	i	Cin	a	b	y	Cout
XOR	0	0	0/X	0	0	0	0
	0	0	0/X	0	1	1	0
	0	0	0/X	1	0	1	0
	0	0	0/X	1	1	0	0
	0	0	1/X	0	0	0	0
	0	0	1/X	0	1	1	0
	0	0	1/X	1	0	1	0
	0	0	1/X	1	1	0	0



Operation	j	i	Cin	a	b	y	Cout
AND	1	0	0/X	0	0	0	0
	1	0	0/X	0	1	0	0
	1	0	0/X	1	0	0	0
	1	0	0/X	1	1	1	0
	1	0	1/X	0	0	0	0
	1	0	1/X	0	1	0	0
	1	0	1/X	1	0	0	0
	1	0	1/X	1	1	1	0

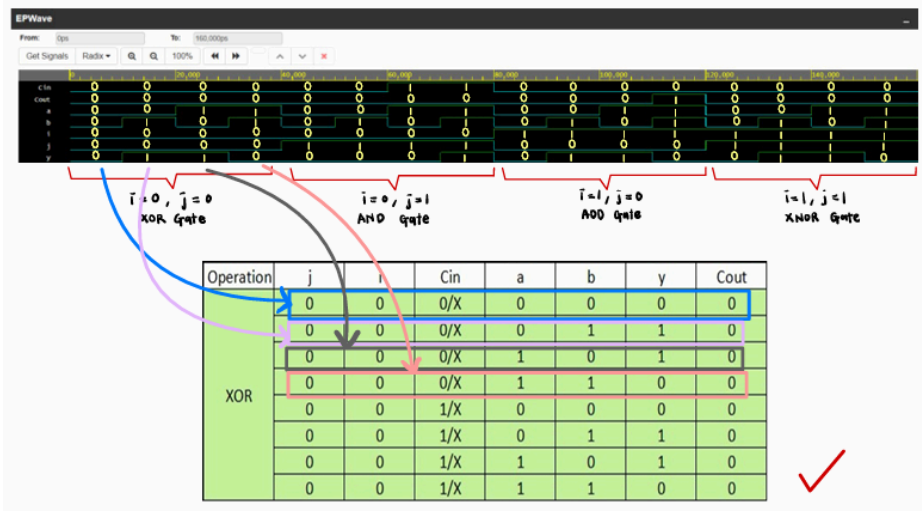


Operation	j	i	Cin	a	b	y	Cout
ADD	0	1	0	0	0	0	0
	0	1	0	0	1	1	0
	0	1	0	1	0	1	0
	0	1	0	1	1	0	1
	0	1	1	0	0	0	0
	0	1	1	0	1	1	0
	0	1	1	1	0	1	0
	0	1	1	1	1	0	1

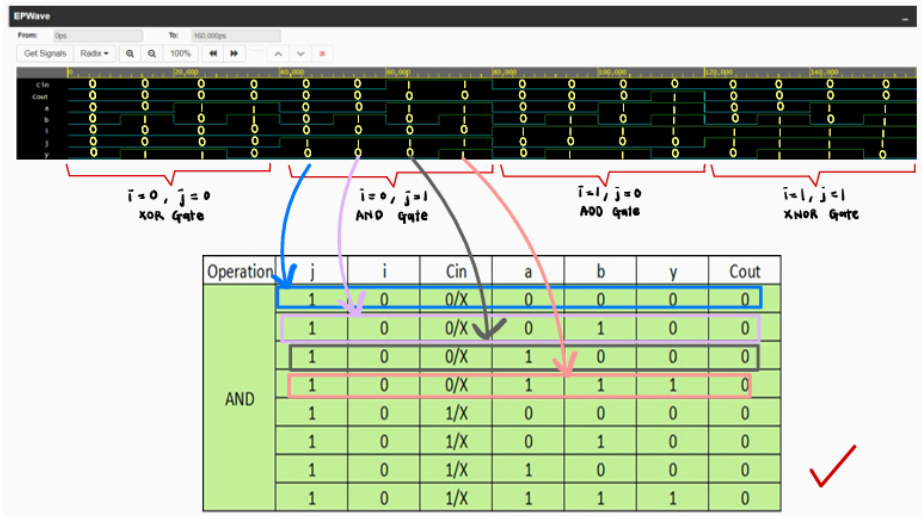


Operation	j	i	Cin	a	b	y	Cout
XNOR	1	1	0/X	0	0	0	0
	1	1	0/X	0	1	1	0
	1	1	0/X	1	0	1	0
	1	1	0/X	1	1	0	0
	1	1	1/X	0	0	0	0
	1	1	1/X	0	1	1	0
	1	1	1/X	1	0	1	0
	1	1	1/X	1	1	0	0

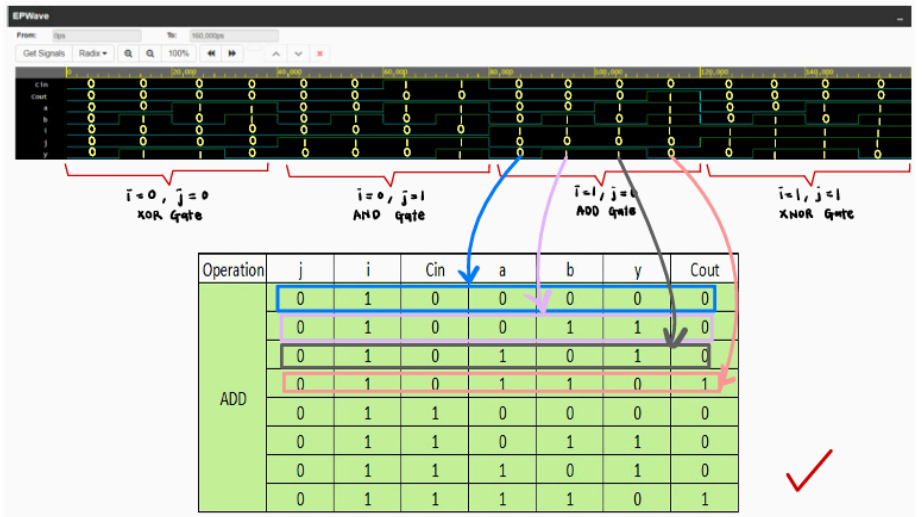
Decoder-Based ALU Waveforms with Truth Table



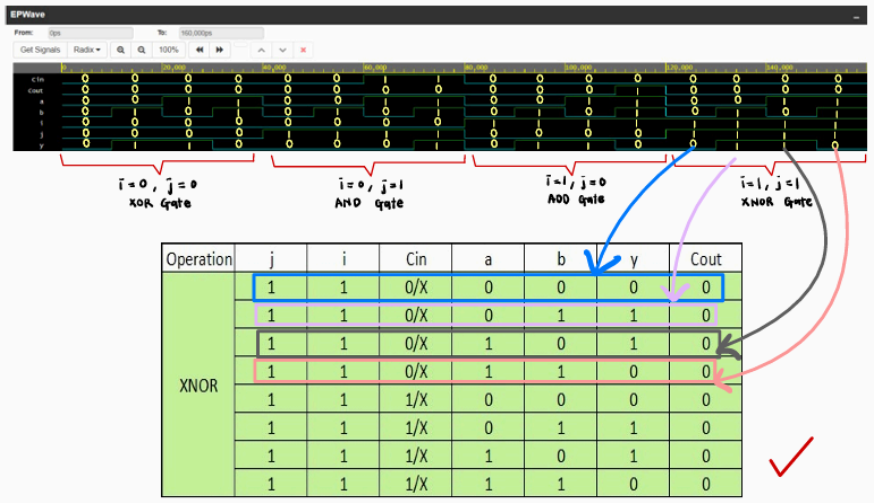
Operation	j	i	Cin	a	b	y	Cout
XOR	0	0	0/X	0	0	0	0
	0	0	0/X	0	1	1	0
	0	0	0/X	1	0	1	0
	0	0	0/X	1	1	0	0
	0	0	1/X	0	0	0	0
	0	0	1/X	0	1	1	0
	0	0	1/X	1	0	1	0
	0	0	1/X	1	1	0	0



Operation	j	i	Cin	a	b	y	Cout
AND	1	0	0/X	0	0	0	0
	1	0	0/X	0	1	0	0
	1	0	0/X	1	0	0	0
	1	0	0/X	1	1	1	0
	1	0	1/X	0	0	0	0
	1	0	1/X	0	1	0	0
	1	0	1/X	1	0	0	0
	1	0	1/X	1	1	1	0



Operation	j	i	Cin	a	b	y	Cout
ADD	0	1	0	0	0	0	0
	0	1	0	0	1	1	0
	0	1	0	1	0	1	0
	0	1	0	1	1	0	1
	0	1	1	0	0	0	0
	0	1	1	0	1	1	0
	0	1	1	1	0	1	0
	0	1	1	1	1	0	1



Operation	j	i	Cin	a	b	y	Cout
XNOR	1	1	0/X	0	0	0	0
	1	1	0/X	0	1	1	0
	1	1	0/X	1	0	1	0
	1	1	0/X	1	1	0	0
	1	1	1/X	0	0	0	0
	1	1	1/X	0	1	1	0
	1	1	1/X	1	0	1	0
	1	1	1/X	1	1	0	0

Based on the comparison of Multiplexer-Based ALU Waveforms and Decoder-Based ALU Waveforms with the Truth Table of 1-bit ALU. It is found that the input (i, j, Cin, a, b) and output (y, Cout) waveforms are the same for both. Given the same waveforms output in the EDA Playground, it shows that there are 2 approaches that can be used to obtain the waveform of 1-bit ALU based on observation, namely using Multiplexer and Decoder.

The waveforms for both approaches are the same because both designs implement similar logical functions based on the truth table of the 1-bit ALU. Although their internal architectures are different, where the Multiplexer acts as a data selector from multiple simultaneous operations, while the Decoder functions as a controller that activates a specific data path, both still map the input combinations a, b, and Cin to identical outputs y and Cout. This similarity proves the concept of functional equivalence in digital systems, which shows that the same design objectives can be achieved through different implementation techniques as long as the underlying logic is consistent.

5.2.3 Pass/Fail Analysis

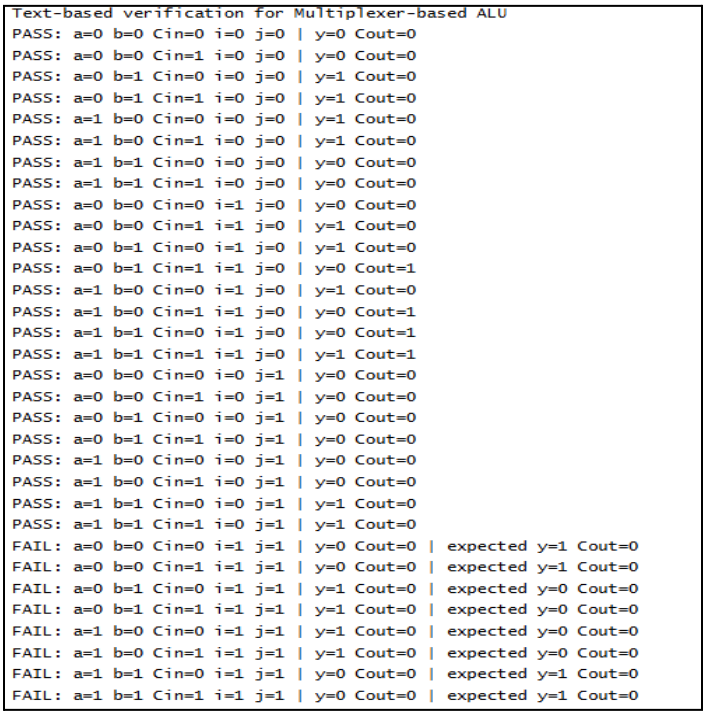


Figure 5.2.3.1: Output after used Alternative Testbench Decoder-based ALU

Figure 5.2.3.1 displays a text validation log showing a comparison between the actual and expected outputs of a decoder-based ALU operation. The results reveal several errors (FAILS) at certain input combinations, indicating the need for improvements to the circuit logic or testbench code used.

To be specific, according to the simulation findings in Figure 5.2.3.1, the system passed the majority of the basic logic and arithmetic operations tests. However, one of the errors (FAIL) was found when the control inputs the values of i=1 and j=1 were typed. This was not successful because the real performance of the circuit was not equal to the actual performance that was expected of the circuit in the testbench. This is probable to be because of a configuration error in the decoder unit that did not trigger the proper module to have the operation code or there

was logic error in the ALU module itself in the input mixture. Additional debugging must occur on the selector logic to make sure that the ALU is able to execute all instruction codes correctly.

```
Text-based verification for Decoder-based ALU
PASS: a=0 b=0 Cin=0 i=0 j=0 | y=0 Cout=0
PASS: a=0 b=0 Cin=1 i=0 j=0 | y=0 Cout=0
PASS: a=0 b=1 Cin=0 i=0 j=0 | y=1 Cout=0
PASS: a=0 b=1 Cin=1 i=0 j=0 | y=1 Cout=0
PASS: a=1 b=0 Cin=0 i=0 j=0 | y=1 Cout=0
PASS: a=1 b=0 Cin=1 i=0 j=0 | y=1 Cout=0
PASS: a=1 b=1 Cin=0 i=0 j=0 | y=0 Cout=0
PASS: a=1 b=1 Cin=1 i=0 j=0 | y=0 Cout=0
PASS: a=0 b=0 Cin=0 i=1 j=0 | y=0 Cout=0
PASS: a=0 b=0 Cin=1 i=1 j=0 | y=1 Cout=0
PASS: a=0 b=1 Cin=0 i=1 j=0 | y=1 Cout=0
PASS: a=0 b=1 Cin=1 i=1 j=0 | y=0 Cout=1
PASS: a=1 b=0 Cin=0 i=1 j=0 | y=1 Cout=0
PASS: a=1 b=0 Cin=1 i=1 j=0 | y=0 Cout=1
PASS: a=1 b=1 Cin=0 i=1 j=0 | y=0 Cout=1
PASS: a=1 b=1 Cin=1 i=1 j=0 | y=1 Cout=1
PASS: a=0 b=0 Cin=0 i=0 j=1 | y=0 Cout=0
PASS: a=0 b=0 Cin=1 i=0 j=1 | y=0 Cout=0
PASS: a=0 b=1 Cin=0 i=0 j=1 | y=0 Cout=0
PASS: a=0 b=1 Cin=1 i=0 j=1 | y=0 Cout=0
PASS: a=1 b=0 Cin=0 i=0 j=1 | y=0 Cout=0
PASS: a=1 b=0 Cin=1 i=0 j=1 | y=0 Cout=0
PASS: a=1 b=1 Cin=0 i=0 j=1 | y=1 Cout=0
PASS: a=1 b=1 Cin=1 i=0 j=1 | y=1 Cout=0
FAIL: a=0 b=0 Cin=0 i=1 j=1 | y=0 Cout=0 | expected y=1 Cout=0
FAIL: a=0 b=0 Cin=1 i=1 j=1 | y=0 Cout=0 | expected y=1 Cout=0
FAIL: a=0 b=1 Cin=0 i=1 j=1 | y=1 Cout=0 | expected y=0 Cout=0
FAIL: a=0 b=1 Cin=1 i=1 j=1 | y=1 Cout=0 | expected y=0 Cout=0
FAIL: a=1 b=0 Cin=0 i=1 j=1 | y=1 Cout=0 | expected y=0 Cout=0
FAIL: a=1 b=0 Cin=1 i=1 j=1 | y=1 Cout=0 | expected y=0 Cout=0
FAIL: a=1 b=1 Cin=0 i=1 j=1 | y=0 Cout=0 | expected y=1 Cout=0
FAIL: a=1 b=1 Cin=1 i=1 j=1 | y=0 Cout=0 | expected y=1 Cout=0
```

Figure 5.2.3.2: Output after used Alternative Testbench Decoder-based ALU

The pass/fail analysis shows that most of the outputs for both the Multiplexer-based and Decoder-based ALUs match the expected results from the truth table, and are therefore marked as PASS. Any discrepancies that occur are identified as FAIL. For example, if Cout appears as 1 when it is not expected, it may indicate a minor connection or logic problem in the ADD module. Overall, the text-based verification confirms that the ALU is functioning properly, as shown in Figure 5.2.3.1 and Figure 5.2.3.2, with Cout only active during ADD operations as needed. A successful self-test provides a clear summary of pass and fail conditions, making functional verification easy and reliable.

6. Results and Discussion

6.1 Functional Correctness Analysis

Based on simulations conducted using a text-based testbench, both the Multiplexer-based ALU and Decoder-based ALU demonstrate correct operation according to the specified truth table. Each input combination was tested and the obtained outputs corresponded to the expected values for XOR, ADD, AND, and XNOR operations. A PASS status is indicated when the output is correct, while any mismatch is marked as FAIL. Overall, no significant functional errors were detected, proving that the ALU design is logically correct.

6.2 Comparison of Multiplexer vs Decoder-Based Architecture

Architecturally, Multiplexer-based ALUs use selectors (select lines) to select one operation output at a time, making them simpler and easier to understand. In contrast, Decoder-based ALUs use decoders to activate only one operation path based on control signals, which provides more orderly control but involves more logical connections. Although both architectures produce the same output, Decoder-based ALUs have a more modular structure and are suitable for additional operation expansion, while Multiplexer-based ALUs are simpler for small-scale designs.

6.3 Performance and Hardware Efficiency Considerations

From a hardware performance and efficiency perspective, Multiplexer-based ALUs tend to use fewer logic gates, thus reducing hardware resource usage and signal delay. This makes them more efficient for simple, high-speed systems. Decoder-based ALUs, on the other hand, require more logic gates such as AND and OR, which can increase hardware usage and slightly increase propagation delay. However, the main advantage of Decoder-based ALUs is their scalability and ease of modification, making them suitable for more complex and flexible systems.

7. Conclusion

Overall, this project successfully achieved its main objective of analyzing the behavior of a 1-bit Arithmetic Logic Unit (ALU) through the study of basic logic and arithmetic operations, and designing two ALU architectures, namely a Multiplexer-based ALU and a Decoder-based ALU using Boolean logic and Verilog HDL. A module-based design was used to ensure a neat and easy to understand system structure. Through simulation, operations such as AND, XNOR, and ADD were thoroughly tested, and the results obtained showed a clear understanding of the principles of digital logic and how arithmetic and combinational logic operations work in a single processing unit for embedded system applications.

In addition, the verification process was successfully implemented through text based testing using simulation result display and direct comparison with truth tables. The simulation results proved that both the Multiplexer ALU and the Decoder ALU functioned accurately for all input combinations tested without any significant logic errors. The comparison of the architectures showed that the Multiplexer ALU is simpler and more efficient in terms of hardware usage, while the Decoder ALU offers a more structured and flexible structure for future operational expansion. Overall, this project not only confirms the functionality of the ALU through truth table analysis and simulation but also provides a solid foundation for the development of more complex digital systems at the next stage.

8. Appendix

A. Testbench Codes

```
`timescale 1ns/100ps
module testbench_text_mux;
    reg a, b, Cin;
    reg i, j;
    wire y, Cout;
    MultiplexerALU DUT (
        .a(a),
        .b(b),
        .Cin(Cin),
        .i(i),
        .j(j),
        .y(y),
        .Cout(Cout)
    );
    reg expected_y, expected_Cout;
    integer errors;
    initial begin
        $display("Text-based verification for Multiplexer-based ALU");
        errors = 0;
        for (integer aj = 0; aj <= 1; aj = aj + 1) begin
            for (integer ai = 0; ai <= 1; ai = ai + 1) begin
                for (integer aa = 0; aa <= 1; aa = aa + 1) begin
                    for (integer ab = 0; ab <= 1; ab = ab + 1) begin
                        for (integer ac = 0; ac <= 1; ac = ac + 1) begin
                            j = aj; i = ai; a = aa; b = ab; Cin = ac;
                            #1;
                            if (j==0 && i==0) expected_y = a ^ b;
                            else if (j==0 && i==1) begin
                                expected_y = (a ^ b) ^ Cin;
                                expected_Cout = (a & b) | ((a ^ b) & Cin);
                            end
                            else if (j==1 && i==0) expected_y = a & b;
                            else expected_y = ~(a ^ b);
                            if (!(j==0 && i==1)) expected_Cout = 0;
                            if (y !== expected_y || Cout !== expected_Cout) begin
                                $display("FAIL: a=%b b=%b Cin=%b i=%b j=%b | y=%b
Cout=%b | expected y=%b Cout=%b",
                                        a,b,Cin,i,j, y,Cout, expected_y, expected_Cout);
                                errors = errors + 1;
                            end else begin
                                $display("PASS: a=%b b=%b Cin=%b i=%b j=%b | y=%b
```

```
Cout=%b",
                                a,b,Cin,i,j, y,Cout);
                        end
                    end
                end
            end
        end
    end

    $display("Text-based verification complete. Total errors: %0d", errors);
    $finish;
  end
Endmodule
```

MultiplexerALU Testbench Code for Text-based verification

```
`timescale 1ns/100ps
module testbench;
  reg a, b, Cin;
  reg i, j;
  wire y, Cout;
  DecoderALU DUT (
    .a(a),
    .b(b),
    .Cin(Cin),
    .i(i),
    .j(j),
    .y(y),
    .Cout(Cout)
  );
  reg expected_y, expected_Cout;
  task compute_expected;
    input a, b, Cin, i, j;
    output y_exp, Cout_exp;
    begin
      case({i,j})
        2'b00: begin // XOR
          y_exp = a ^ b;
          Cout_exp = 0;
        end
        2'b01: begin // ADD
          {Cout_exp, y_exp} = a + b + Cin;
```

```
            end
            2'b10: begin // AND
               y_exp = a & b;
               Cout_exp = 0;
            end
            2'b11: begin // XNOR
               y_exp = ~(a ^ b);
               Cout_exp = 0;
            end
         endcase
      end
   endtask
   initial begin
      $display("a b Cin i j | y Cout | expected_y expected_Cout | Status");
      $dumpfile("decoderALU_text.vcd");
      $dumpvars(0, testbench);
      integer ia, ib, iCin, ii, ij;
      for (ia=0; ia<2; ia=ia+1)
         for (ib=0; ib<2; ib=ib+1)
            for (iCin=0; iCin<2; iCin=iCin+1)
               for (ii=0; ii<2; ii=ii+1)
                  for (ij=0; ij<2; ij=ij+1) begin
                     a = ia; b = ib; Cin = iCin; i = ii; j = ij;
                     #1;
compute_expected(a,b,Cin,i,j,expected_y,expected_Cout);

                     if (y === expected_y && Cout === expected_Cout)
                        $display("%b %b %b %b %b | %b %b | %b %b | PASS",
                              a,b,Cin,i,j, y,Cout, expected_y, expected_Cout);
                     else
                        $display("%b %b %b %b %b | %b %b | %b %b | FAIL",
                              a,b,Cin,i,j, y,Cout, expected_y, expected_Cout);
                  end
      $finish;
   end
endmodule
```

DecoderALU Testbench Code for Text-based verification